Figure 14.17 Running Dijkstra's algorithm on a directed graph (derived from the example in figure 14.9).

This is due to a fundamental difference: BFS only uses the number of edges traversed as a metric, and if we use edge's weight instead, then it's possible that a path including more edges has a lower weight than another path including fewer edges: for instance, looking at figure 14.17, there are two paths between v_1 and v_2; the path $v_1 \rightarrow v_3 \rightarrow v_2$ only traverses 2 edges, but its total weight is 8, while the other path, $v_1 \rightarrow v_3 \rightarrow v_5 \rightarrow v_2$ has length 3, but its total weight is just 5. The second path is longer, and visits one more vertex between v_3 and v_2, so the distance to v_2 will be initially set to 8 (when v_3 is visited) and then updated to 5, when it's v_5's turn to be visited.

Because of this behavior, every time a new vertex is visited, potentially all its neighbors' priorities can be updated.

In turn, this influences the asymptotic performance of Dijkstra's algorithm, which depends on how efficiently this "update priority" operation can be implemented. In details, for Dijkstra, we can implement the priority queue as:

- An array (sorted or unsorted, as described in appendix C);
- A heap;
- A *Fibonacci* heap.

The running time using an array for the priority queue is going to be $O(|V|^2)$, because each vertex will require